DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: **04/11/2026**

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Introduction and Problem Definition

The hospital stores discharge summaries, prescriptions, operation notes and nursing observations as free text. A clinician searching for a word such as “fracture” may miss records containing “fractures”, “fractured” or “fracturing”. Medical terms may also appear as abbreviations, spelling variants and different date or dosage formats. The proposed system creates a finite-state text-processing layer before machine learning: it recognises structured entities, tokenises and normalises text, analyses morphology, and maps related surface forms to searchable canonical forms.

The design uses exact rules where the format is predictable, a lexicon and finite-state transducer for morphology, and a separate noise-handling layer for OCR and code-mixed text.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis (3 marks)

1.1 Levels of linguistic analysis

Level	Meaning	Hospital example and relevance
Phonology	Sound patterns of a language.	Not required for ordinary typed-text search. It becomes

		relevant indirectly for speech recognition, not part of this text-only baseline.
Morphology	Internal structure of words and affixes.	Essential: fracture/fractures/fractured; nephrectomy/nephrectomies.
Syntax	How words form phrases and sentences.	Limited use: identify structured phrases such as “500 mg twice daily”; full parsing is not required for basic indexing.
Semantics	Meaning of words and expressions.	Important for synonym relationships such as kidney/renal and for avoiding clinically unsafe conflation.
Pragmatics	Meaning in context and speaker intention.	Useful for interpreting negation and context: “no fracture” must not be treated as a positive fracture case.
Discourse	Meaning across sentences and document structure.	Useful when linking a diagnosis in one sentence to its treatment or outcome later in the record.

The core first-stage system requires morphology and structured-text recognition. Syntax, semantics, pragmatics and discourse become increasingly important for high-quality clinical retrieval. Phonology can safely be excluded from the initial typed-text system, but OCR and speech extensions may require sound-related processing.

1.2 Three model families

Model family	**Best hospital task**	**Why it fits / why alternatives are weaker**
State machines (DFA/NFA/FST)	Validate identifiers, dates, blood pressure formats and map word forms to analyses.	They give explicit transitions and predictable recognition. A probabilistic model is unnecessary for strict formats; hand-written rules alone do not naturally represent a running state transition system.
Formal rule systems	Case folding, abbreviation expansion and spelling rules such as y-to-i or e-deletion.	Rules are transparent and maintainable for known transformations. A state machine alone does not express the full rationale of a sequence of linguistic rewrite rules as clearly; probabilities are unnecessary for deterministic spelling rules.
Probabilistic models	Rank uncertain OCR corrections or ambiguous candidate interpretations.	They can rank alternatives using evidence. Exact rules are better for fixed formats, while a

		deterministic automaton cannot rank uncertain alternatives.

1.3 Ambiguity in the corpus

Ambiguity	Example	Problem
Lexical	“lead” may mean a metal or to guide; “mass” may mean a lesion or a quantity.	One spelling has multiple meanings.
Morphological	“saw” may be a past-tense verb or a noun. “findings” contains a plural suffix.	A surface form may have more than one morphological analysis.
Segmentation	“500mg” versus “500 mg”; “post-op” versus “post op”.	Token boundaries are not always explicit.
Abbreviation	“OD” may mean once daily or right eye depending on context. “BP” means blood pressure.	The same short form can have different expansions.

Ambiguity is central because a text string does not always determine one meaning or one analysis. The system must preserve alternatives when context is insufficient and avoid silently converting an uncertain interpretation into a clinical fact.

1.4 Search and resource costs

- Automata state-space search: a DFA follows one transition per input symbol; an NFA may maintain a set of active states. DFA recognition is O(n) time for an input of length n and O(1) working state apart from the automaton. An NFA simulation can require $O(n \times |Q|)$ time and $O(|Q|)$ active-state memory.
- Regex backtracking: a backtracking engine may explore alternative paths. Ambiguous nested quantifiers can cause excessive or even exponential time on adversarial strings. Use bounded patterns, anchors and unambiguous constructions.
- Annotated data: deterministic pattern development needs examples and error labels; probabilistic ranking needs substantially more representative labelled examples.
- Expert effort: clinicians or trained annotators must validate abbreviations, drug terminology, morphology and relevance judgements. Lexicon maintenance is a continuing cost.

1.5 Proposed pipeline

Hospital records → Regex entity recognition → Tokenisation → Normalisation → DFA/NFA validation → Lexicon + FST morphological analysis → Porter fallback for unknown English forms → Search index → Clinician query and ranked records

Input: de-identified free-text records. Output: searchable records with recognised entities, canonical tokens and morphology annotations. Each stage stores the original text so that normalisation never destroys the source record.

TASK 2 – Basic Regular Expression Patterns (4 marks)

2.1 Basic operators

Operator	Meaning	Clinical example
Concatenation	Patterns occur in sequence.	AB matches “BP”.
Disjunction |	Either alternative.	(mg|mcg)
Kleene star *	Zero or more repetitions.	[0-9]*
Plus +	One or more repetitions.	[0-9]+
Optional ?	Zero or one occurrence.	-? for optional minus sign.
Character class []	One character from a set.	[0-9]
Range	A continuous character range.	[A-Z]
Negated class [^]	Any character except listed ones.	[^\s]+
Anchors ^ and $	Start and end of string.	^[0-9]{10}$
Counters {m,n}	Bounded repetition.	[0-9]{2,4}

2.2 Twelve entity patterns

The following Python patterns are practical starting rules. Drug names should be checked against a maintained drug lexicon; a regex alone cannot reliably recognise every drug or distinguish all clinical contexts.

Entity	Regex (Python raw string)	Must match / must not match
Patient identifier	\\b(?:PID|MRN)[-]?[A-Z0-9]{6,12}\\b	Matches PID-AB123456 or MRN 12345678; rejects shorter IDs and unrelated words.
Admission date	\\b(?:0?[1-9]|[12][0-9]|3[01])[-/](?:0?[1-9]|1[0-2])[-/](?:19|20)\\d{2}\\b	Matches DD/MM/YYYY or DD-MM-YYYY; rejects impossible month numbers. Calendar validity needs date parsing.
Discharge date	\\b(?:discharged|discharge)\\s*(?:on:?)?\\s*(\\d{1,2}[-/]\\d{1,2}[-/](?:19|20)\\d{2})\\b	Matches a date explicitly introduced by discharge; rejects dates not labelled as discharge dates.
Drug + strength	\\b[A-Za-z][A-Za-z-]{2,}(?:\\s+[A-Za-z][A-Za-z-]{1,})?\\s+\\d+(?:\\.\\d+)?\\s*(?:mg|mcg|g|ml|mL|IU)\\b	Matches “paracetamol 500 mg”; rejects a bare number without a recognised unit. Confirm drug name with lexicon.
Dosage frequency	\\b(?:once|twice|thrice)\\s+daily\\b|\\b(?:OD|BD|BID|TDS|TID|QID|q\\d+h)\\b	Matches common frequency expressions; rejects arbitrary abbreviations. Context must disambiguate OD.
Lab value + unit	\\b[A-Za-z][A-Za-z0-9-]{1,}\\s*[:=]?\\s*\\d+(?:\\.\\d+)?\\s*(?:mg/	Matches “Hb 12.5 g/dL”; rejects unsupported units.

	dL|g/dL|mmol/L|mEq/L|U/L|%|10\\^?\\d+/L)\\b	
Blood pressure	\\b(?:BP\\s*)?\\d{2,3}\\s*/\\s*\\d{2,3}\\s*mmHg\\b	Matches “BP 120/80 mmHg”; rejects missing unit in this strict version.
Temperature	\\b(?:Temp(?:erature)?\\s*)?\\d{2}(?:\\.\\d)?\\s*°?\\s*[CF]\\b	Matches “Temp 37.0 C”; rejects values without C/F. Clinical range validation is separate.
ICD code	\\b[A-TV-Z][0-9][A-Z0-9](?:\\.[A-Z0-9]{1,4})?\\b	Matches common ICD-10-like codes; code-set/version validation is required.
Ward and bed	\\b(?:Ward|W)\\s*[A-Z0-9-]+\\s*[,/-]?\\s*(?:Bed|B)\\s*[A-Z0-9-]+\\b	Matches “Ward A, Bed 12”; rejects ward-only strings.
Contact number	\\b(?:\\+91[-]?)?[6-9]\\d{9}\\b	Matches Indian 10-digit mobile number with optional +91; rejects numbers starting 0–5.
Age and symptom duration	\\b\\d{1,3}\\s*(?:years?|yrs?|months?|days?|weeks?)\\s*(?:old)?\\b|\\b(?:for\\s*)?\\d+\\s*(?:hours?|days?|weeks?|months?)\\b	Matches “45 years old” or “for 3 days”; a contextual label is needed to distinguish age from duration.

2.3 Greedy and non-greedy matching

```
import re\ns = "Give 500 mg paracetamol and 250 mg amoxicillin."\nprint(re.findall(r".*mg", s))     # greedy: longest span ending at the last mg\nprint(re.findall(r".*?mg", s))   # non-greedy: shortest span ending at each mg
```

The greedy .* expands as far as possible before allowing “mg” to match, so it can capture from the beginning through the second dosage. The non-greedy .*? expands only as far as needed and can return the first dosage span, then continue to find the next. For extracting each dosage safely, use a bounded expression such as r"\b\d+(?:\.\d+)?\s*mg\b".

2.4 Precedence and grouping

```
Incorrect:  ^Ward|Bed[0-9]+$\nCorrect:    ^(?:Ward|Bed)[0-9]+$
```

Alternation has low precedence. The incorrect expression means “starts with Ward” OR “Bed followed by digits at the end”, not “the whole string is either Ward plus digits or Bed plus digits”. Group alternatives and anchor the complete expression.

2.5 Evaluation

For each pattern, annotate the expected spans in the corpus and compare predicted spans. TP = correctly identified entities; FP = predicted entities that are not entities; FN = missed true entities. Precision = TP/(TP+FP), recall = TP/(TP+FN), and F1 = 2PR/(P+R). Common FP causes include ambiguous abbreviations, a non-drug word before a strength, and a date-like number that is not a date. Common FN causes include OCR errors, unseen abbreviations, punctuation variation and unsupported units. Actual counts must be calculated from the corpus.

TASK 3 – Regex Tokenisation and Normalisation (4 marks)

3.1 Tokenizer

```
TOKEN_RE = re.compile(
   r"""(?:[A-Za-z]+(?:\.[A-Za-z]+)+\.?       # abbreviations e.g. Dr., i.v.
   |[A-Za-z]+(?:-[A-Za-z]+)+                 # hyphenated terms
   |[0-9]+(?:\.[0-9]+)?(?:mg|mcg|ml|mL|g)?   # number with optional attached unit
   |[A-Za-z]+                                # ordinary word
   |[^\w\s])""", re.VERBOSE | re.IGNORECASE
)
tokens = TOKEN_RE.findall(text)
```

Maximal munch selects the longest valid token at the current position. “post-operative” is one hyphenated token; “Dr.” is retained as an abbreviation; “12.5mg” is retained as one numeric-unit token. A whitespace split would incorrectly keep punctuation attached and would not separate or consistently identify these forms.

Input	Whitespace split	Proposed tokens
Dr. Kumar reviewed the post-operative note.	Dr. | Kumar | reviewed | the | post-operative | note.	Dr. | Kumar | reviewed | the | post-operative | note | .
Hb 12.5g/dL was recorded.	Hb | 12.5g/dL | was | recorded.	Hb | 12.5 | g/dL | was | recorded | . (extend unit alternative to keep g/dL together)
BP: 120/80 mmHg	BP: | 120/80 | mmHg	BP | : | 120/80 | mmHg

3.2 Normalisation rules and abbreviations

```
import re
ABBR = {
 "BP":"blood pressure", "HR":"heart rate", "RR":"respiratory rate",
 "Temp":"temperature", "SOB":"shortness of breath", "CP":"chest pain",
 "NPO":"nothing by mouth", "IV":"intravenous", "IM":"intramuscular",
 "PO":"by mouth", "OD":"once daily", "BD":"twice daily",
 "BID":"twice daily", "TDS":"three times daily", "TID":"three times daily",
 "QID":"four times daily", "PRN":"as needed", " stat":"immediately",
 "Dx":"diagnosis", "Hx":"history", "FH":"family history",
 "PMH":"past medical history", "HTN":"hypertension",
 "DM":"diabetes mellitus", "MI":"myocardial infarction",
 "UTI":"urinary tract infection", "CXR":"chest X-ray",
 "ECG":"electrocardiogram", "CBC":"complete blood count",
 "WBC":"white blood cell", "RBC":"red blood cell",
 "Hb":"haemoglobin", "Cr":"creatinine", "Na":"sodium",
 "K":"potassium", "Sx":"symptoms", "Tx":"treatment"
}
def normalise(text):
   text = text.casefold()
   # expand only as whole tokens; clinical context may still be needed
   for short, full in sorted(ABBR.items(), key=lambda x: -len(x[0])):
```

```
        text = re.sub(r"(?<!\\w)" + re.escape(short.casefold()) + r"(?!\\w)",
                      full, text)
    text = re.sub(r"(?i)(\\d{1,2})[-.](\\d{1,2})[-.](\\d{4})",
                  r"\\3-\\2-\\1", text)  # example DD-MM-YYYY -> YYYY-MM-DD
    text = re.sub(r"(?i)(\\d+(?:\\.\\d+)?)\\s*milligrams?\\b",
                  r"\\1 mg", text)
    text = re.sub(r"(?i)(\\d+)\\s*micrograms?\\b", r"\\1 mcg", text)
    return text
```

The abbreviation list is a starter inventory of more than 30 forms. It must be reviewed by clinicians because short forms such as OD, CP and MS can have multiple meanings. Expansion should be context-aware and should preserve the original text.

3.3 Full illustrative record before and after

Before: “MRN AB123456. 45 yrs male admitted on 03/04/2026 with SOB for 2 days. BP 120/80 mmHg. Hb 12.5 g/dL. Rx: Paracetamol 500mg PO BD. Discharged on 08/04/2026.”

After (illustrative normalisation): “mrn ab123456. 45 years male admitted on 2026-04-03 with shortness of breath for 2 days. blood pressure 120/80 mmhg. haemoglobin 12.5 g/dl. prescription: paracetamol 500 mg by mouth twice daily. discharged on 2026-04-08.”

The date example assumes day-month-year input. Ambiguous dates such as 03/04/2026 must not be converted without a documented hospital convention.

3.4 Irreversible decisions

- Case folding removes original capitalisation. Usually acceptable for case-insensitive search, but not for case-sensitive identifiers or gene symbols.
- Abbreviation expansion replaces a short form with one interpretation. It is unsafe when an abbreviation is ambiguous; retain the original and store the expansion as an alternative.
- Removing punctuation or hyphens can merge distinct identifiers or change meaning. Preserve the raw form alongside the normalised form.
- Converting units can introduce rounding. Store the original value and unit as well as any converted value.

A regular expression cannot recognise arbitrarily nested balanced parentheses. Balanced nesting requires memory proportional to nesting depth; a finite automaton has only finitely many states and cannot count an unbounded number of opening brackets and match them later.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence (5 marks)

4.1 Formal definition

A finite automaton is a five-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the initial state and F is the set of accepting states. A DFA

has exactly one next state for each state-symbol pair. An NFA may have zero, one or several next states and may use ε-transitions.

4.2 Four DFA validators

For compact diagrams, each chain below has a start state, one state after each consumed character position and a final accepting state. A dead state handles every invalid transition. These are schematic automata; the alphabet is partitioned into the character classes shown.

Validator	State sequence / transition idea	Accepted example
10-digit mobile	q0 --[6-9]→ q1 --[0-9]→ q2 ... --[0-9]→ q10 (final); all other transitions → dead.	9876543210
Patient ID PID- plus 6 alphanumerics	q0-P-q1-I-q2-D-q3-optional separator state-then six to twelve alphanumeric transitions to final states.	PID-AB1234
BP value	Digit states for 2–3 systolic digits → slash → 2–3 diastolic digits → " mmHg" → final.	120/80 mmHg
ICD-like code	Letter state → digit state → alphanumeric state → optional dot and 1–4 alphanumeric states.	S72.001A

A complete implementation uses explicit transition tables for each character class, with a dead state for all unspecified transitions. The Python validator below implements exact full-string acceptance with compiled regexes; a classroom DFA diagram should expand each repeated position into a separate state.

4.3 NFA with epsilon transition: optional country prefix

Consider a mobile number written either as 9876543210 or +91 9876543210. The NFA branches at the start: one path skips the country prefix using ε; the other consumes "+91 ". Both paths then consume ten digits, with the first digit restricted to 6–9.

> NFA start q0: ε→q1 (no prefix), and q0 --+→q2 --9→q3 --1→q4 --space→q1.\nq1 --[6-9]→q5; q5 --[0-9]→q6→q7→...→q14 (final).

Accepted trace for 9876543210: ε-closure(q0) = {q0,q1}; read 9 gives {q5}; read each remaining digit advances one state; after the tenth digit the active set contains q14, so accept. Rejected trace for 5876543210: the first digit 5 has no transition from q1, so the active set becomes empty and the string is rejected.

4.4 Subset construction and minimisation

Subset construction converts an NFA to a DFA. Each DFA state is a set of NFA states. Start with ε-closure({q0}); for each input symbol a, compute ε-closure(move(S,a)). A subset is accepting if it contains at least one NFA final state.

DFA subset	Input digit class	Next subset	Accept?
{q0,q1}	6–9	{q5}	No
{q0,q1}	0–5	∅	No
{q5}	0–9	{q6}	No
{q6}	0–9	{q7}	No
...	0–9	next position state	No
{q14}	—	—	Yes
∅	any	∅	No

The full subset table for this linear-prefix NFA contains the reachable prefix subsets plus the dead subset. Minimisation merges states only when no possible remaining suffix distinguishes them. For a fixed-length ten-digit suffix, positions 1 through 10 are distinguishable because the number of digits remaining differs; the dead state is also distinct. The exact state count depends on whether the prefix branch is represented with a shared start state and whether equivalent prefix states are merged. Record the actual count after running a standard partition-refinement algorithm; do not claim an unverified count.

4.5 Regex and automata equivalence

Thompson construction converts a regular expression into an ε-NFA by creating fragments for symbols, concatenation, alternation and Kleene star. For example, regex [6-9][0-9]{9} becomes a chain of ten consuming transitions, with the first transition labelled 6–9 and the remaining nine labelled 0–9. Conversely, state elimination derives a regex: eliminate intermediate states while updating $R_{ij} := R_{ij} \mid R_{ik}(R_{kk})^*R_{kj}$. The resulting expression for this chain is [6-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9], equivalent to [6-9][0-9]{9}.

4.6 Pumping lemma: balanced parentheses are not regular

Pumping lemma: if L is regular, there is a pumping length p such that every string s in L with $|s| \geq p$ can be written s = xyz, where $|xy| \leq p$, $|y| > 0$, and $xy^iz \in L$ for every $i \geq 0$.

Assume $L = \{(^n)^n : n \geq 0\}$ is regular and let p be its pumping length. Choose s = "(" repeated p times followed by ")" repeated p times. Since $|xy| \leq p$, the non-empty y consists only of opening parentheses. Pumping down with i=0 removes some opening parentheses but leaves all p closing parentheses, producing unequal counts. The result is not in L, a contradiction. Therefore L is not regular. This explains why regex/finite automata cannot validate arbitrary nested balanced brackets.

TASK 5 – Inflectional Morphology (3 marks)

5.1 Definitions and distinction

Term	Definition
Morpheme	Smallest unit that carries grammatical or lexical meaning.
Stem	Base to which an affix is attached.
Affix	Bound morpheme attached before, after or within a base.

Inflection	Creates a grammatical form without changing the basic word class or core lexical identity.
Paradigm	Set of grammatical forms belonging to one lexeme.

Inflection generally preserves word category and is productive: fracture → fractures. Derivation may create a new lexeme or change category/meaning: infect → infection. Derivational affixes are often closer to the root than inflectional endings; English plural -s normally occurs outside derivational suffixes.

5.2 Inflectional paradigms and examples

Lexeme	Noun number	Verb forms	Other forms
fracture	fracture / fractures	fracture / fractures / fractured / fracturing	—
admit	admission (derivation); patient / patients	admit / admits / admitted / admitting	—
treat	treatment (derivation)	treat / treats / treated / treating	—
infect	infection (derivation); infection / infections	infect / infects / infected / infecting	—
severe	—	—	severe / severer / severest (comparison; often “more/most severe”)

The corpus should be checked for noun plural, third-person singular agreement, past tense, past participle, present participle and adjective comparison. Not every paradigm will occur in every record type.

5.3 English spelling rules

Rule	Implementation idea	Over-generation counter-example
e-deletion	Drop final silent e before a vowel-initial suffix: create + ing → creating.	“be” + ing does not become “bing”; exceptions and lexical restrictions are needed.
Consonant doubling	Double final consonant for a stressed CVC base before a vowel suffix: admit + ed → admitted.	“visit” → visitted is wrong; stress and syllable conditions matter.
y-to-i	Consonant + y changes y to i before -es/-ed: study → studies/studied.	Vowel + y: play → plays/played, not plaies.
Sibilant -es	Add -es after s, x, z, ch, sh: box → boxes.	Not every word ending in a similar sound follows a simple spelling rule; lexical exceptions exist.

5.4 Irregular forms

Maintain an explicit irregular table, for example: be → am/is/are/was/were/been; have → has/had; go → went/gone; do → does/did/done; child → children; foot → feet; tooth → teeth; diagnosis → diagnoses; analysis → analyses. These forms cannot all be generated reliably by the regular suffix rules, so the lexicon stores them directly.

5.5 Measuring inflection effect

Type-to-lemma ratio = number of distinct surface word types divided by number of distinct lemmas. A higher ratio indicates more surface variation per lemma. For the 40-query experiment, mark each failed query and determine whether an inflection-only mismatch explains it. Inflection-explained failure proportion = inflection-only failed queries / all failed queries × 100. The assignment does not supply the corpus or relevance labels, so the numerical values must be computed after annotation.

TASK 6 – Derivational Morphology (4 marks)

6.1 Derivational affixes

Affix	Example	Typical contribution / category
-itis	arthritis	noun-forming: inflammation
-ectomy	nephrectomy	noun-forming: surgical removal
-ology	cardiology	noun-forming: field of study
-osis	fibrosis	noun-forming: condition/process
-aemia	anaemia	noun-forming: blood condition
-ation	inflammation	noun-forming: process/result
-ment	treatment	noun-forming: result/process
-er	examiner	noun-forming: person/agent
-able	treatable	adjective-forming: capable of being treated
-ive	protective	adjective-forming: having a quality/function
-al	renal	adjective-forming: relating to
-ic	cardiac	adjective-forming: relating to
nephro-	nephrology	prefix: kidney-related
cardio-	cardiology	prefix: heart-related
hyper-	hypertension	prefix: excessive/high
hypo-	hypoglycaemia	prefix: low/under
re-	reoperation	prefix: again
un-	untreated	prefix: not/reversal

6.2 Derivational families and order

Root	Family / structure	Comment
infect	infect → infection → infectious	Root + nominal suffix; adjective suffix attaches to a

		suitable base.
operate	operate → operation → operational	Verb → noun → adjective.
treat	treat → treatment → treatable	Verb → noun/adjective relatives.
nephr	nephro- + logy → nephrology; nephro- + ectomy → nephrectomy	Combining form precedes the medical suffix.
cardi	cardio- + logy → cardiology; cardi- + ac → cardiac	Medical combining form builds related terms.
fracture	fracture → fractures; fracture → fractured	Plural and tense are inflection, not derivation.

Impossible or ill-formed orders include "-ectomynephro" (the suffix cannot normally precede the combining form) and "-abletreat" (English -able is a suffix, not a prefix). "hyper-itis" is not a normal formation; the prefix combines with an appropriate base, e.g. hyperglycaemia.

6.3 Non-compositionality and search conflation

A derived clinical term may have a specialised meaning that cannot be calculated by simply combining affix meanings. "Inflammation" is a medical process, but a specific diagnosis such as "appendicitis" names a defined condition. "Nephrectomy" denotes surgical removal of a kidney, not merely any kidney-related event. Aggressive stemming may incorrectly merge words that are medically distinct.

Conflation helps	**Why**	**Conflation can mislead**	**Why**
fracture / fractures	Same diagnosis with number variation.	infection / infectious	Condition and descriptive property are not identical.
nephrectomy / nephrectomies	Singular/plural forms of the same procedure.	hypertension / hypotension	Opposite clinical conditions must remain distinct.

6.4 Productivity

For each affix, count distinct derived word types and total token occurrences in the corpus. An affix is productive when it appears with several different bases and in newly formed or less frequent words. It is fossilised when it occurs in a small set of fixed lexical items. The actual type/token counts cannot be supplied without the corpus; report the measured counts from the analysis script rather than invented values.

TASK 7 – Finite-State Morphological Parsing (5 marks)

7.1 FST definition

A finite-state transducer is a finite-state machine whose transitions read symbols from an input tape and emit symbols on an output tape. In morphology, the lexical tape contains a lemma plus grammatical features, while the surface tape contains the written word. For example, lexical "fracture

+ N + PL" maps to surface "fractures". The relation can be used in either direction: analysis reads surface and returns lexical analyses; generation reads a lexical form and produces a surface form.

7.2 Continuation-class design

LEXICON Noun: fracture → NounInflection; kidney → NounInflection; diagnosis → IrregularPlural\nNounInflection: +SG → ε; +PL → s (or spelling-rule path)\nLEXICON Verb: admit → VerbInflection; treat → VerbInflection\nVerbInflection: +PRES3SG → s; +PAST → ed; +PROG → ing\nLEXICON Adjective: severe → AdjInflection\nAdjInflection: +COMP → er; +SUPER → est

A production lexicon should contain at least 300 medically reviewed stems, 20 inflectional affixes and 15 derivational affixes, with irregulars stored separately. The above is a small explanatory specification, not a claim that the required 300-stem resource has already been built.

7.3 Worked surface analyses

Surface form	Lexical analysis	Morphological features
fractures	fracture + N + PL	noun, plural
fractured	fracture + V + PAST	verb, past tense (context-dependent)
fracturing	fracture + V + PROG	verb, progressive/participle
patients	patient + N + PL	noun, plural
admitted	admit + V + PAST	verb, past tense; spelling doubles t
diagnoses	diagnosis + N + PL	irregular/orthographic plural

Trace example for fractures: lexical tape consumes f-r-a-c-t-u-r-e followed by +N +PL; the transducer maps the lemma to the surface base "fracture" and the plural feature to "s"; output is "fractures". Parsing follows the same arcs in reverse relation: "fractures" → "fracture +N +PL".

7.4 Generation and ambiguity

Generation uses the same relation in the opposite direction: fracture +N +PL → fractures; admit +V +PAST → admitted. The transducer is a relation, not two unrelated rule sets, so the same mapping supports both directions.

Ambiguous examples: "treated" can be analysed as treat + V + PAST or treat + V + PAST_PARTICIPLE. "wound" can be wound + N + SG or wind + V + PAST. The analyser should return all valid analyses. A downstream contextual component uses sentence syntax and nearby words to rank them; it must retain uncertainty if context is insufficient.

TASK 8 – Orthographic Rules and Two-Level Morphology (4 marks)

8.1 Why the lexicon alone is insufficient

- admit + ed by naive concatenation gives admited; correct surface form is admitted.
- study + s gives studys; correct form is studies.

- create + ing gives createing; correct form is creating.

8.2 Rewrite rules

Rule	Rewrite notation / condition	Example
Silent-e deletion	e → 0 / stem __ +V-suffix	create + ing → creating
Consonant doubling	C → CC / stressed CVC __ +V-suffix	admit + ed → admitted
Y-to-I	y → i / C __ + s or ed	study + es → studies
Sibilant plural	ε → e / sibilant __ + s	box + s → boxes
Past tense after e	ε → d / e __ + PAST	love + ed → loved

8.3 Cascade and two-level approach

Lexical input (lemma + features) → Lexicon FST → Morphological intermediate form → Orthographic rule transducers → Surface word

A sequential cascade applies one transducer after another. Rule order matters: identify the morpheme boundary and suffix first, then apply stem spelling changes, then final suffix realisation. Koskenniemi two-level morphology instead constrains lexical/surface symbol pairs in parallel using contextual rules. A cascade is modular and easier to debug; two-level rules can express interacting spelling constraints without forcing one fixed sequence, but may be harder to author and troubleshoot.

Test at least twenty forms spanning all rules, including create/creating, admit/admitted, study/studies, box/boxes, love/loved and irregular forms. The report should show expected form, generated form and pass/fail. Do not mark unexecuted tests as successful.

TASK 9 – Porter Stemmer (5 marks)

9.1 Definitions

Porter stemming is a suffix-stripping algorithm for English. A consonant is a letter that is not a, e, i, o or u; y is treated contextually (a consonant when preceded by a vowel, otherwise a vowel). A word can be represented as [C](VC)^m[V], where m is the measure: the number of vowel-consonant sequences. The condition *v* means the stem contains a vowel; *d means the stem ends in a double consonant; *o means the stem ends in consonant-vowel-consonant where the final consonant is not w, x or y.

9.2 Illustrative measure calculations

Word	V/C pattern (simplified)	m (illustrative)
fractures	C C V C C V C V C	count VC transitions using Porter grouping; calculate mechanically
treated	C C V V C V C	depends on stem after suffix removal
patients	C V C V V C C	depends on y/vowel grouping
admitted	V C C V C V C	depends on suffix stripping
infection	V C C V C V C	calculate on candidate stem
renal	C V C V C	one VC sequence

For a formally correct report, calculate m using the Porter consonant/vowel definition on the stem at each step, not by counting ordinary syllables. The following code uses NLTK's Porter implementation, which is suitable for reproducible output; the hand-worked intermediate traces should be generated from the exact implementation version used.

9.3 Five stages and clinical caution

- Step 1a: remove plural endings, e.g. -sses, -ies, -s under Porter conditions.
- Step 1b: handle -eed/-ed/-ing and post-processing such as adding e or doubling a consonant.
- Step 1c: change terminal y to i when the stem contains a vowel.
- Step 2: replace longer derivational suffixes, e.g. -ational → -ate, under measure conditions.
- Step 3–5: further suffix replacement/removal and final e/l handling.

```
from nltk.stem import PorterStemmer\nps = PorterStemmer()\nwords = ["fractures", "fractured", "fracturing", "patients", "admitted", "infection"]\nfor w in words:\n    print(w, "->", ps.stem(w))
```

Stemming is not lemmatisation. A stemmer may return a non-word stem; a morphological analyser returns a dictionary lemma and features. Over-stemming merges distinct clinical terms; under-stemming fails to merge genuine variants. Evaluate both on the same 200 hand-annotated forms.

For retrieval, run each of the 40 relevance-judged queries in three conditions: no normalisation, Porter stemming and full morphological analysis. For each condition calculate precision, recall and F1 over the judged relevant documents. The uploaded assignment does not include those judgments, so the table must be filled after running the experiment.

TASK 10 – Implement and Test the Pipeline (3 marks)

10.1 Python implementation (standard library + NLTK)

```
import re, time
from collections import Counter
from nltk.stem import PorterStemmer

TOKEN_RE = re.compile(r"[A-Za-z]+(?:-[A-Za-z]+)*|\d+(?:\.\d+)?(?:mg|mcg|ml|mL|g)?|[^\w\s]", re.I)
PATTERNS = {
 "mobile": re.compile(r"[6-9]\d{9}"),
 "patient_id": re.compile(r"(?:PID|MRN)[- ]?[A-Z0-9]{6,12}", re.I),
 "bp": re.compile(r"(?:BP\s*)?\d{2,3}\s*/\s*\d{2,3}\s*mmHg", re.I),
 "icd": re.compile(r"[A-TV-Z][0-9][A-Z0-9](?:\.[A-Z0-9]{1,4})?")
}
def tokenize(text):
    return TOKEN_RE.findall(text)

def recognise(text):
    found = {}
    for name, pat in PATTERNS.items():
        found[name] = pat.findall(text)
    return found
```

```
def normalise_tokens(tokens):
    return [t.casefold() for t in tokens]

porter = PorterStemmer()
def stem_tokens(tokens):
    return [porter.stem(t) for t in tokens if re.fullmatch(r"[a-z]+", t)]

def process_record(text):
    t0 = time.perf_counter()
    toks = tokenize(text)
    entities = recognise(text)
    norm = normalise_tokens(toks)
    stems = stem_tokens(norm)
    elapsed = time.perf_counter() - t0
    return {"tokens": toks, "entities": entities,
            "normalised": norm, "stems": stems, "seconds": elapsed}

if __name__ == "__main__":
    sample = ("MRN AB123456. BP 120/80 mmHg. "
              "Paracetamol 500mg twice daily. Patient has fractures.")
    result = process_record(sample)
    print(result)
```

This is a compact executable baseline. Extend PATTERNS to all twelve patterns, add the complete abbreviation table, lexicon, irregular dictionary, FST relation and explicit DFA/NFA transition structures before claiming that the full assignment implementation is complete.

10.2 Required test strategy

Test group	Positive test	Negative test / check
Each regex pattern	One or more valid clinical examples.	At least one near-miss or malformed example per pattern.
Automata	Valid identifier, phone, BP and ICD strings.	Wrong first digit, missing separator, too few digits or invalid symbol.
Morphology	fractures, admitted, diagnoses.	Unknown stem and irregular not covered by a rule.
Ambiguity	treated and wound return multiple analyses.	Confirm all valid analyses are preserved.
Pipeline	One record from each of four record types.	Malformed and empty records.

For every stage record: number of records/tokens processed, number matched or analysed, number rejected, elapsed time and software version. Use time.perf_counter() around each stage. The required plot can show runtime against record length or analyser coverage against lexicon size. A real plot requires actual run outputs.

10.3 Metrics

```
precision = TP / (TP + FP)\nrecall    = TP / (TP + FN)\nF1        = 2 * precision * recall / (precision + recall)\ncoverage
= analysed_tokens / total_tokens
```

Use zero-division handling when a denominator is zero. Report micro-averaged and, where useful, per-entity/per-record-type results. Split evaluation data from development data to avoid tuning patterns on the final test set.

TASK 11 – Compare, Recommend and Defend the Design (2 marks)

Strategy	Retrieval behaviour	Coverage / speed	Maintenance
No normalisation	Exact forms only; misses inflectional variants.	Fast and simple but low recall for variant-heavy queries.	Very low.
Porter stemmer	Improves recall for many English suffix variants; may over-stem.	Fast, no lexicon required; broad English coverage but weak clinical precision.	Low.
Finite-state morphological analysis	Returns lemma and grammatical features; can preserve ambiguity and irregulars.	High precision for covered vocabulary; unknown words reduce coverage.	High initial lexicon/rule effort and ongoing clinical review.

Recommendation: deploy a hybrid finite-state design. Use regex and DFA for structured entities, a reviewed lexicon/FST for known clinical morphology, and Porter stemming only as a low-confidence fallback for unknown English words. Keep original tokens and surface forms. Do not conflate semantically different derivational relatives automatically. Use clinician-reviewed synonyms as a separate semantic expansion layer.

The final deployment choice should be supported by measured precision, recall, F1, coverage, latency and maintenance hours. A learned model becomes more economical when vocabulary variation, OCR noise and language mixing grow beyond what manual lexicon and rule maintenance can cover, provided sufficient representative labelled data and clinical governance are available.

TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt (8 marks)

A. Impact Analysis (2 marks)

The changed input includes OCR character confusions (0/O, 1/l, rn/m), line-break hyphenation, missing diacritics, Romanised regional-language tokens, code-mixing and at least 20% out-of-lexicon, misspelt or non-English tokens. The original assumptions of clean typed English, closed vocabulary and exact accept/reject matching no longer hold.

Exact regexes and deterministic automata will reject noisy identifiers and dates. A closed English lexicon will fail to analyse regional-language forms. The analyser coverage will fall as the out-of-lexicon rate rises. If 20% of tokens are outside the supported vocabulary, coverage cannot exceed 80%

for those tokens unless correction, fallback or vocabulary expansion recovers some of them. This is an upper-bound illustration, not a measured result.

B. Evaluation of the Original Design (2 marks)

Record type	Illustrative input	Where original pipeline fails
OCR	"MRN AB12O456; fract ure noted; BP 12O/8O mmHg."	Identifier and BP exact patterns reject O/0 substitutions; split "fract ure" is not recognised as fracture.
Code-mixed	"patient ku fever irundhuchu, paracetamol 500mg kuduthen."	English morphology does not analyse Tamil romanisation; "ku", "irundhuchu" and "kuduthen" are out of lexicon.
Both	"pat1ent ku fev er irundhuchu; paracetamoI 5OOmg kuduthen."	OCR substitutions, broken tokens, regional morphology and drug spelling noise cause entity and analyser failures.

Run these records through the baseline and count true entities and analyses manually. Compare noisy-sample precision/recall, analyser coverage and retrieval F1 with the clean baseline. Those figures are not available in the assignment source and must be measured; do not fabricate them.

Weighted edit-distance matching is suitable for likely OCR confusions because substitutions such as O/0 and l/1 can receive lower costs. A character n-gram language identifier can help identify English versus Romanised regional-language tokens. Both should generate candidates or language labels, not silently overwrite clinical data.

C. Adaptation or Redesign (2 marks)

Decision: MODIFY the original design and supplement it with uncertainty-aware components. Retain the core finite-state layer because exact structured formats and regular morphology remain valuable.

1. OCR correction front end: use weighted edit distance or a weighted finite-state transducer to generate candidate corrections. Apply character-confusion costs and preserve the original OCR string.
2. Line-break repair: join end-of-line hyphenation only when dictionary and layout evidence support it; retain uncertain alternatives.
3. Token-level language identification: use character n-gram evidence to label English, Romanised Tamil and unknown tokens.
4. Lexicon expansion: add clinician-reviewed regional-language stems, transliteration variants and code-mixed expressions. Store language and source metadata.
5. Fallback: use Porter only for likely English out-of-lexicon words; do not apply it to Romanised Tamil.
6. Entity recognition: retain strict DFA validation for clean candidates and add weighted candidate matching for noisy OCR. Require confirmation for high-risk identifiers, dates and dosages.
7. Rule cascade: add OCR repair before tokenisation, then tokenisation, language identification, normalisation, entity validation and morphological analysis.

Expected effect: coverage should improve for recoverable OCR and known regional forms; precision may decrease if candidate correction is too permissive. Runtime and annotation effort will increase because candidate ranking, lexicon maintenance and review are added. Quantify the effect on the evaluation sample before deployment.

D. Evidence-Based Defence (2 marks)

Modification is the best decision because the original finite-state design remains effective for clean, well-formed input but exact matching cannot handle noisy OCR and open-vocabulary code-mixing. Replacing the whole system would discard transparent, efficient validators and hand-built morphology that still work. A hybrid design preserves deterministic recognition where possible and uses weighted matching and language identification only where uncertainty occurs.

The defence must cite measured evidence from the project: (1) pattern precision/recall on clean and noisy records, (2) analyser coverage on English and code-mixed tokens, (3) retrieval F1 under the three normalisation strategies, (4) runtime per record and (5) hours spent on lexicon and annotation. Without the dataset, the correct conclusion is qualitative and the numeric fields remain pending measurement.

Remaining limitations: finite-state methods do not understand unrestricted meaning, resolve all clinical ambiguity, infer missing context, or model arbitrarily nested structures. They can encode many regular patterns but cannot by themselves provide full semantic interpretation or guarantee safe clinical decisions. Human review and clinical governance remain necessary.

Conclusion

The proposed hospital search layer combines regular expressions, finite-state automata, morphological resources and a Porter fallback. Regex and DFA provide precise structured-entity recognition; tokenisation and normalisation standardise text; an FST maps surface forms to lemmas and grammatical features; and stemming offers a fast fallback for selected unknown English forms. For OCR and Romanised code-mixing, the system should be modified with weighted edit-distance correction, token-level language identification and an expanded multilingual lexicon. The original text must always be retained, and all clinical normalisation decisions must be validated by domain experts.

References

- Jurafsky, D. and Martin, J. H. Speech and Language Processing. Draft 3rd edition. Chapters on regular expressions, finite-state morphology and information retrieval.
- Manning, C. D., Raghavan, P. and Schütze, H. Introduction to Information Retrieval. Cambridge University Press, 2008.
- Porter, M. F. “An algorithm for suffix stripping.” Program, 14(3), 1980, pp. 130–137.
- Koskenniemi, K. Two-Level Morphology: A General Computational Model for Word-Form Recognition and Production. University of Helsinki, 1983.
- Bird, S., Klein, E. and Loper, E. Natural Language Processing with Python. O’Reilly Media, 2009.